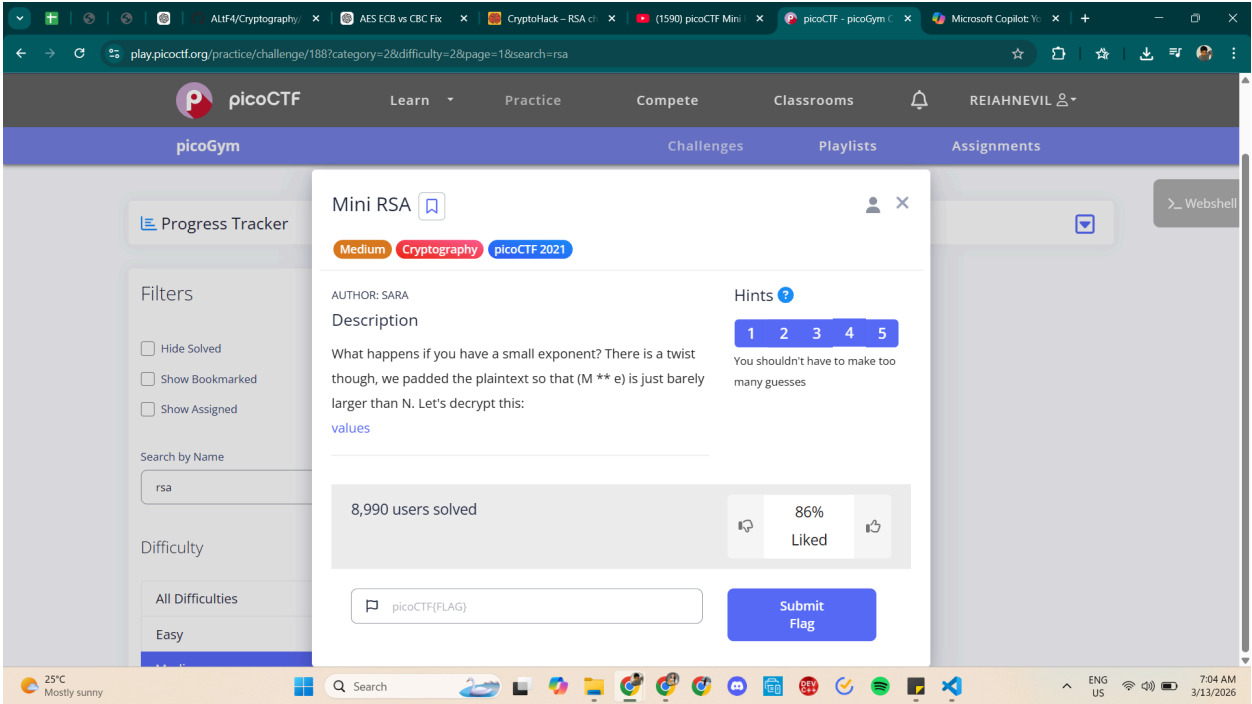


PRACTICAL ACTIVITY 1 [CHALLENGE 1]: Small Exponent Attack Write-up



CHALLENGE TITLE: Mini RSA
(<https://play.picoctf.org/practice/challenge/188?category=2&difficulty=2&page=1&search=mini%20rsa>)
PLATFORM: picoCTF
CATEGORY: Cryptography (Medium)

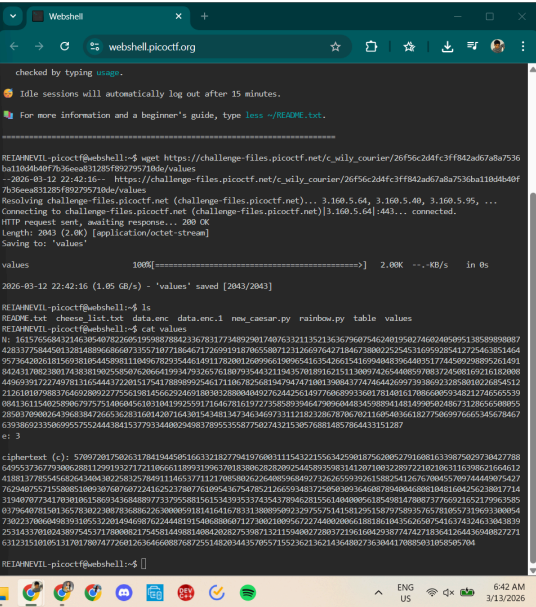
Challenge Overview:

The challenge provided the following RSA parameters:

- Modulus N
- Public exponent e = 3
- Ciphertext c

*The description hinted: What happens if you have a small exponent? There is a twist though, we padded the plaintext so that (M ** e) is just barely larger than N.*

This indicates that the challenge involves a small public exponent vulnerability in RSA.



PROCESS:

When I started the challenge, I first looked at the RSA parameters that were given: the modulus **N**, the ciphertext **c**, and the public exponent **e** = 3. Since **e** = 3 is a small exponent, I remembered that this can sometimes lead to weaknesses in Cryptography if the message is not handled carefully. Normally, RSA encryption computes $c = M^e \bmod N$. If M^e were smaller than **N**, then the ciphertext would just be M^e , which would make it easy to recover the plaintext by

taking the cube root. However, the challenge description said that M^e was **barely larger than N** . This made me think about how modular arithmetic works. Since the ciphertext is the remainder after dividing M^e by N , it means that M^e can be written as $M^e = c + kN$ for some integer k . Based on this idea, I tried iterating over possible values of k and computing the cube root of $c + kN$. I wrote a Python script (`mini_rsa.py`) that loops through values of k , uses the `iroot` function to check if the cube root is exact, and then converts the result into bytes to see if it contains the picoCTF flag format. After running the script, one of the values produced a perfect cube and decoding it revealed the flag. This approach worked because the plaintext cube was only slightly larger than N , so only a small number of k values needed to be tested before recovering the original message.

```
mini_rsa.py X
WEEK-3 > mini_rsa.py > ...
1 from gmpy2 import iroot
2 from binascii import unhexlify
3
4 ciphertext = 570972017502631784194450516633218277941976003111543221556342590187562005279160816339875029730427788649553736779300628811299193271721106
5 n = 161576568432146305407822605195988788423367831773489290174076332113521363679607546240195027460240509513858989808742833775844501328148896686607335
6 e = 3
7
8 for k in range(0, 1000000):
9     m, exact = iroot(ciphertext + k*n, e)
10    if exact:
11        hex_m = format(m, 'x')
12
13        if len(hex_m) % 2:
14            hex_m = '0' + hex_m
15
16        try:
17            message = unhexlify(hex_m)
18        except:
19            continue
20
21        if b"pico" in message:
22            print("FLAG:", message.decode())
23            break
```

PS C:\CYBSEC\WEEK-3> python mini_rsa.py

FLAG: picoCTF{e_sh0u1d_b3_lArg3r_92f4d5a5}

Learn

Practice

Compete

Classrooms

REIAHNEVIL

picoGym

Challenges

Playlists

Assignments

Progress Tracker

Filters

Search by Name

Difficulty

Mini RSA

Medium Cryptography picoCTF 2021

AUTHOR: SARA

Description

What happens if you have a small exponent? There is a twist though, we padded the plaintext so that (M^e) is just barely larger than N . Let's decrypt this:

8,990 users solved

86% Liked

Submit Flag

Hints

1 2 3 4 5

You shouldn't have to make too many guesses

picoCTF{e_sh0u1d_b3_lArg3r_92f4d5a5}

25°C Mostly sunny

Search

ENG US

7:04 AM 3/13/2026

Hurray! You solved this challenge.

Challenges

Play